Hedge Fund Panel Forecasting

Multi-Horizon Time-Series Modelling under a Weighted Skill Score Metric

Author 1 Author 2 Author 3 Author 4

DA-IICT, Semester 6
Applied Forecasting Methods, 2026

Abstract

Abstract to be written after final results have landed. This project studies a panel of approximately 37,000 anonymised time series taken from a Kaggle hedge-fund forecasting competition. The target is forecast at four lookahead horizons under a weighted skill score that concentrates roughly two thirds of the evaluation mass in the top one percent of rows. The first phase of the work, presented in this draft, establishes the structure of the panel, the diagnostic conditions for classical and machine-learning models, and three findings that were not surfaced in the midway report: the four horizons are coupled cumulations of a single short-step process, the maximum panel weight lives at horizon three, and per-horizon target scaling differs by a factor of more than four.

1 Problem Statement

We work with a publicly released Kaggle competition dataset titled *Hedge fund – time-series forecasting*. The competition sponsor releases a panel of anonymised numerical series with engineered features and asks competitors to forecast a continuous target at four lookahead horizons. The dataset is intentionally anonymised: identifiers, feature names, and the underlying financial domain are stripped. What remains is a clean, mathematically well-defined supervised forecasting problem with a non-trivial weighting scheme.

1.1 What is being forecast

Each row of the panel is identified by the tuple

$$(\texttt{code},\ \texttt{sub\_code},\ \texttt{sub\_category},\ \texttt{horizon},\ \texttt{ts\_index})$$

and carries a continuous target y_{target}, a non-negative competition weight, and 86 anonymised numerical covariates of the form $\texttt{feature\_a}, \texttt{feature\_b}, \ldots$ The integer time index `ts_index` runs over the contiguous range 1–3601 in the public training partition and 3602–4376 in the held-out test partition. The training panel contains 5,337,414 rows and the test panel contains 1,447,107 rows.

The four forecast horizons are $H \in \{1,\ 3,\ 10,\ 25\}$, interpreted as the lookahead in time units. For each $(\texttt{code}, \texttt{sub\_code}, \texttt{sub\_category})$ the panel typically carries all four horizons at the same `ts_index`, so the natural unit of analysis is the 4-tuple key above. There are 36,923 distinct series in train under that definition.

1.2 Why the problem is hard

The dataset combines three well-known difficulties that make naive baselines a bad guide:

1. **Heavy-tailed target.** The target distribution is centred near zero with an interquartile range of approximately ± 0.05, but the full range runs from roughly -2200 to $+2300$. The 1% and 99% percentiles are at approximately ± 82. Plain mean-squared losses are therefore extremely sensitive to a small number of tail rows.

2. **Concentrated evaluation weight.** The competition metric is sample-weighted, and the weights are extraordinarily skewed: the median weight is approximately 1.7×10^3 while the maximum is 1.4×10^{13}. As a result, the top one percent of rows by weight carry approximately 64% of all evaluation mass, and the top five percent carry approximately 92%. Any unweighted analysis can recommend the wrong model.

3. **Limited per-series history at long horizons.** Although the panel is large in aggregate, only about 5% of individual series cleanly cross the validation cutoff used in this work (`ts_index` $= 2880$). Some series have as few as seven training points before the cutoff, which makes per-series classical fitting infeasible for many model families.

1.3 Why the problem is interesting

Beyond being a real competitive forecasting problem, three properties make the dataset a useful subject for a methodological course project:

- *Anonymisation forces statistical reasoning.* Without semantic feature names we cannot fall back on domain priors. Every claim must be supported by a measurable property of the data.

- *The weighting scheme rewards precision on rare rows.* A model that improves average behaviour while degrading on the high-weight rows can lose to a simpler model. This is a mechanically different objective from standard regression and warrants explicit study.

- *The four horizons are coupled.* As we show in Section 4.14, the targets at $H \in \{3, 10, 25\}$ are well approximated by rolling cumulative sums of the $H = 1$ process. This structure is not advertised in the dataset description and admits a modelling strategy that the midway report did not consider.

1.4 Scope of this report

The remainder of this report covers the diagnostic and exploratory phase of the project. Section 2 formalises the panel keys and the chronological constraint that any validation strategy must respect. Section 3 develops the official competition metric in detail, with worked numerical examples and a comparison against standard error metrics. Section 4 carries out a full exploratory analysis: schema sanity, missingness, target and weight distributions, stationarity diagnostics, autocorrelation structure, feature audit, and three findings about the multi-horizon structure of the panel that constrain every modelling decision in subsequent chapters of the project. Modelling chapters (classical baselines, global gradient-boosted models, the multi-horizon aggregation experiment, probabilistic forecasting, and ensembles) will be added as those notebooks land.

2 Forecast Targets and Series Identity

A panel forecasting problem only makes sense once the unit of analysis is unambiguous. This section fixes the definitions used throughout the project.

2.1 Panel keys

A row of the panel has primary keys (`code`, `sub_code`, `sub_category`, `horizon`, `ts_index`). We treat the 4-tuple

$$s \equiv (\texttt{code}, \texttt{sub\_code}, \texttt{sub\_category}, \texttt{horizon})$$

as the natural *series identifier*, and treat `ts_index` as the time axis. There are 23 codes, 180 distinct sub-codes, 5 sub-categories, and 4 horizons in train, yielding 36,923 distinct series. Test uses a narrower sub-code support of 47 distinct values, which is a coverage shift rather than a schema mismatch (every test category level already exists in train).

2.2 The four horizons are coupled

A naive reading of the schema would treat the four horizons of the same (`code`, `sub_code`, `sub_category`) as four independent series, simply distinguished by an extra integer label. Two empirical facts in Section 4 contradict this view:

1. For approximately 1.21 million distinct (`code`, `sub_code`, `sub_category`, `ts_index`) tuples, all four horizons are simultaneously present (Section 4.13).

2. For the same triple, the target at horizon k is closely approximated by the rolling sum of the $H = 1$ target over the next k steps:

$$y_t^{H_k} \approx \sum_{i=0}^{k-1} y_{t+i}^{H_1}.$$

   We verify this on a stratified sample of 500 triples in Section 4.14, and find median Pearson correlations of 0.96, 0.94 and 0.91 at $k = 3, 10, 25$ respectively.

The practical consequence is that fitting four independent regressors per triple discards a strong constraint that the data construction itself imposes. We exploit this constraint in the H1-aggregation experiment scheduled for a later notebook.

2.3 Chronological constraint

The training partition runs over `ts_index` $\in [1, 3601]$ and the test partition over $[3602, 4376]$. Because test is a strict temporal suffix of train, any internal validation strategy must also be chronological, with all training points strictly preceding the validation points in time. Random or leave-one-series-out cross-validation would mix future information into earlier states and overstate generalisation.

We use a single canonical chronological partition throughout the project, fixed in the shared utilities at `cf.splits`:

- **Train:** `ts_index` ≤ 2880.
- **Meta** (used only for fitting ensemble weights): $2880 <$ `ts_index` ≤ 3240.
- **Holdout** (final scored window): $3240 <$ `ts_index` ≤ 3601.
- **Test** (`ts_index` ≥ 3602): unlabeled, used only for submission-style sanity checks.

The cutoff `ts_index` $= 2880$ matches the cutoff used in the midway report, preserving comparability with prior diagnostic figures. The meta/holdout boundary at 3240 places approximately one third of the post-cutoff window into the meta slice, leaving the remaining two thirds for final scoring.

2.4 Eligibility for per-series studies

Some classical model families (ARIMA, ARMA, MA, AR with order ≥ 2) require a minimum training history. For per-series diagnostic studies we restrict attention to series that both cross the train/validation cutoff and have at least 120 training rows. There are 1,930 such *eligible* series in the panel, roughly 5% of the 36,923 total. Global pooled models and the H1-aggregation experiment use every row whose `ts_index` falls inside the relevant split, so they are not constrained by per-series length.

3 The Weighted Skill Score

The competition is decided by a single scalar: the *weighted skill score*. Because the rest of the project uses this metric for every model comparison, ensemble weight, and bootstrap interval, it deserves a dedicated section. We state the formula, give the intuitive reading, and note the two practical consequences that carry through every modelling decision.

3.1 Definition

Given truths y_i, predictions $\hat{y}_i$, and non-negative weights w_i for $i = 1, \ldots, n$, the weighted skill score is

$$\text{score}(y, \hat{y}; w) \;=\; \sqrt{1 \;-\; \text{clip}_{[0,1]}\left(\frac{\sum_i w_i \,(y_i - \hat{y}_i)^2}{\sum_i w_i \, y_i^2}\right)} \tag{1}$$

where $\text{clip}_{[0,1]}(x) = \min(\max(x, 0), 1)$. The function returns 1 for a perfect prediction and 0 when the prediction is no better than the all-zero forecast.

3.2 Intuitive meaning

Strip away the symbols and the score answers a single question: *how much of the weighted target "energy" is left unexplained by the prediction?*

The fraction inside the square root, $\sum w_i (y_i - \hat{y}_i)^2 \,/\, \sum w_i y_i^2$, is exactly that — the unexplained share of weighted squared truth. It is zero when the prediction is perfect and one when the prediction is identically zero. The clip caps it at one so that no model can score worse than the trivial all-zero forecast, and the outer square root puts the result on a familiar correlation-like scale where 1 is excellent, 0 is no skill, and intermediate values read roughly the way a Pearson correlation of the same magnitude reads.

A few useful reference points:

- **Skill** $= 1$ — perfect predictions on every weighted row.
- **Skill** $= 0$ — the model is no better than predicting zero. This is the natural "did you learn anything" threshold.
- **Skill** ≈ 0.5 — roughly three quarters of the weighted target energy is still unexplained. In feel, this is comparable to a Pearson correlation of 0.5: a real but modest signal.
- **Skill** ≈ 0.95 — only ten percent of energy unexplained. Strong, near the practical ceiling for a competitive entry.

The reason the metric is built around *weighted* energy rather than raw error is that hedge-fund-style panels have wildly different target scales across instruments. Dividing by $\sum w_i y_i^2$ neutralises that scale: a 1% relative error on a low-volatility row and a 1% relative error on a high-volatility row contribute proportionally to the score. The weights w_i themselves let the organiser encode importance — and in our panel those weights are extraordinarily concentrated, so the score is effectively decided by how well the model fits a small set of high-weight rows.

3.3 Practical consequences for this project

Equation (1) is implemented once in the shared utility `cf.metrics.weighted_skill_score` and used by every later notebook for both model selection and final scoring. The function `cf.metrics.bootstrap_skill` returns 95% confidence intervals by row resampling; we report bootstrap intervals alongside every leaderboard number to avoid declaring winners that are within sampling noise of each other.

Two structural observations carry through to every subsequent modelling decision:

- **High-weight rows decide the leaderboard.** Because approximately 64% of weight sits in the top 1% of rows (Section 4.4), the score is essentially a measure of how well the model predicts those few rows. We must therefore train with sample weights, not just evaluate with them.
- **Plain RMSE will lie.** A model that improves average behaviour at the cost of a small degradation on a high-weight row can lose to a simpler model. We never report unweighted RMSE as a primary headline number.

4 Dataset and Exploratory Analysis

This section reports the exploratory analysis of the panel before any modelling. The aim is twofold. First, to establish the basic statistical shape of the data so that later modelling choices can be motivated rather than guessed. Second, to surface three structural findings about the panel — multi-horizon coupling, the cumulation hypothesis, and per-horizon weight dominance — that the midway report did not document but that materially change the modelling space. All numerical results below are produced by notebook `00_data_structure.ipynb` and the corresponding figures are saved as both PNG and PDF assets.

4.1 Schema, panel scale, and train–test comparability

The training panel contains 5,337,414 rows, organised by 23 distinct `code` values, 180 distinct `sub_code` values, and 5 `sub_category` values across 4 horizons. The integer time index `ts_index` runs over the contiguous range $[1, 3601]$. The test partition continues from `ts_index` $= 3602$ to 4376 with 1,447,107 rows, so test is a strict temporal suffix of train. Treating the 4-tuple (`code`, `sub_code`, `sub_category`, `horizon`) as the natural series identifier yields 36,923 distinct series.

The only columns present in train but not in test are y_{target} and `weight`, as expected. Every category level seen in test exists in train, so there is no cold-start schema issue. The sub-code support narrows from 180 in train to 47 in test — a coverage shift rather than a schema mismatch.

4.2 Missing values

Missingness is computed on a 500,000-row stratified sample to keep memory in check. Of the 86 anonymised features, 48 contain at least one missing value, but the maximum missing rate across all columns is approximately 12.5%. Figure 1 shows the top fifteen offenders.

Reading. Missing rates are bounded; within-fold median imputation suffices. No separate imputation strategy is developed.

4.3 Target distribution

The target y_{target} is centred very close to zero (median -0.0006, IQR ± 0.05), but has extremely heavy tails. The 1% and 99% quantiles sit at approximately -82.8 and $+62.9$ respectively, while the full range runs from approximately -2202 to $+2314$.

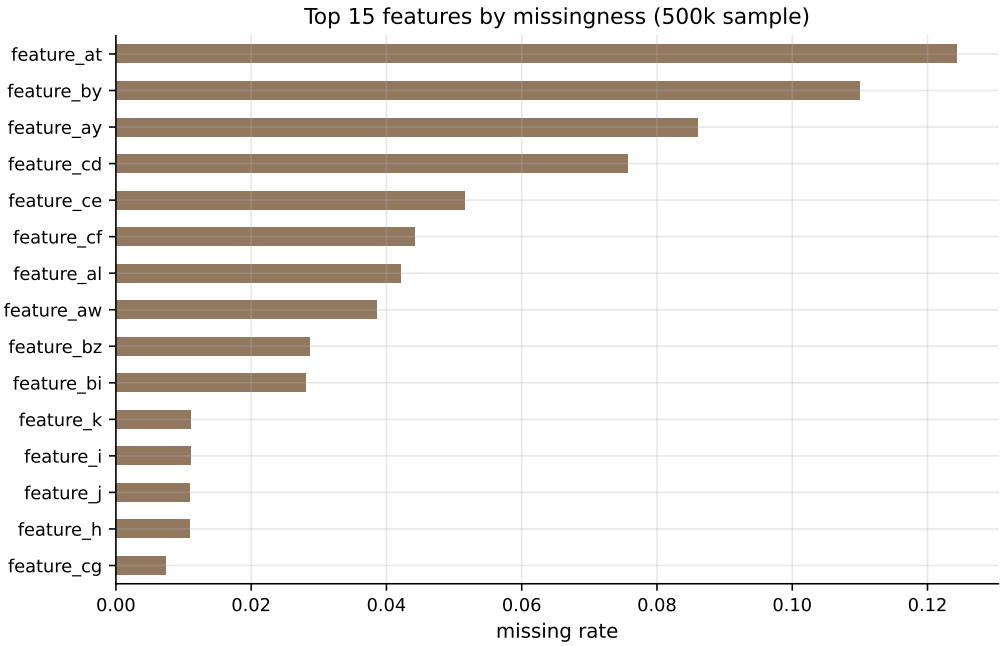

Figure 1: Top fifteen features by missingness, on a 500,000-row stratified sample of train. The maximum missing rate is approximately 12.5%, low enough that median imputation inside a fold is sufficient and there is no need to drop columns or use complex imputers.

Reading. The shape disqualifies plain unweighted MSE as a sensible loss. Tails are real outcomes, not corruption, so we cannot simply clip them; combined with the weighting scheme below, this is the central methodological constraint.

4.4 Weight distribution and metric concentration

The competition weights are extraordinarily skewed. The median weight is 1,699, while the maximum is 1.39×10^{13}. Figure 3 shows the weight distribution clipped at the 99th percentile (left) and the cumulative-share Lorenz-style curve (right). The top 1% of rows carry 64.2% of total weight; the top 5% carry 92.6%; the top 10% carry 98.3%.

Reading. Unweighted evaluation describes a different competition. The weighted skill score (Section 3) is our only headline metric, and a model that accurately predicts the top one percent of rows has more leverage on the leaderboard than a uniformly mediocre one.

4.5 Series coverage and split feasibility

Not every series has enough history to support time-based validation. Figure 4 shows the distribution of per-series lengths. Out of 36,923 distinct series, only 1,930 (approximately 5%) cleanly cross the train/validation cutoff at `ts_index` = 2880. Many series end before the cutoff, contributing zero validation rows.

Reading. Per-series classical fitting is feasible only on the eligible subset (cross the cutoff with at least 120 training rows). Global pooled models are not length-constrained — one reason the comparison between local classical and global ML approaches is not perfectly apples-to-apples.

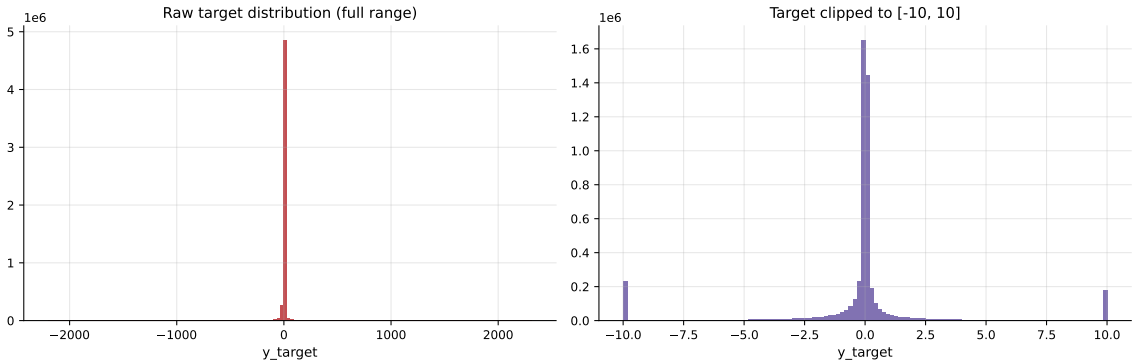

Figure 2: Raw target distribution (left) and the same distribution clipped to $[-10, 10]$ (right). Most of the mass concentrates near zero, with rare extreme tails up to the order of ± 2200.

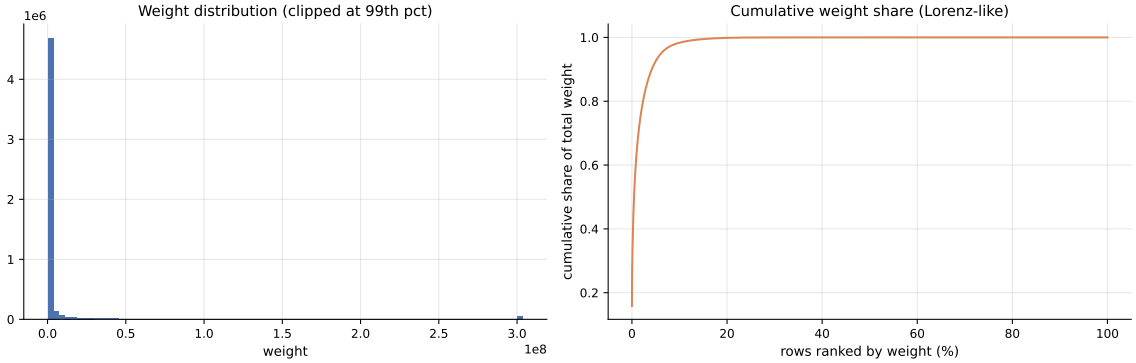

Figure 3: Weight distribution clipped at the 99th percentile (left) and cumulative share of total weight ranked by weight (right). Approximately 64% of total weight is concentrated in the top 1% of rows; 92% in the top 5%.

4.6 Representative-series selection

For every per-series figure in this project we plot the same four series. They are selected with explicit reasons rather than at random, in order to span four distinct stress regimes: long history, large weight, high variability, and near-flat behaviour. Table 1 lists the four chosen series. They match the selection used in the midway report, preserving visual continuity.

Table 1: Four representative series chosen for per-series studies. Each series probes a different regime of the panel.

Reason	Series key (code/sub_code)	Horizon	Length
Longest history	X9BZ68VQ / OYJGNSQK / DPPUO5X2	1	212
Highest total weight	SJZP0OVU / OYJGNSQK / NQ58FVQM	25	156
Most volatile	W4S29LF4 / KL66VIS3 / PHHHVYZI	25	162
Most stable	SJZP0OVU / OYJGNSQK / NQ58FVQM	1	170

Figure 5 shows the raw y_{target} trajectory of each chosen series with the train/validation cutoff marked. The four panels look profoundly different: the most-volatile series swings between -500 and $+500$, while the most-stable series occupies a numerical range narrower than 0.001.

Reading. A one-size-fits-all model will struggle: the most-stable series is essentially numerical noise, while the most-volatile series has obvious cyclical structure. Forecast performance across

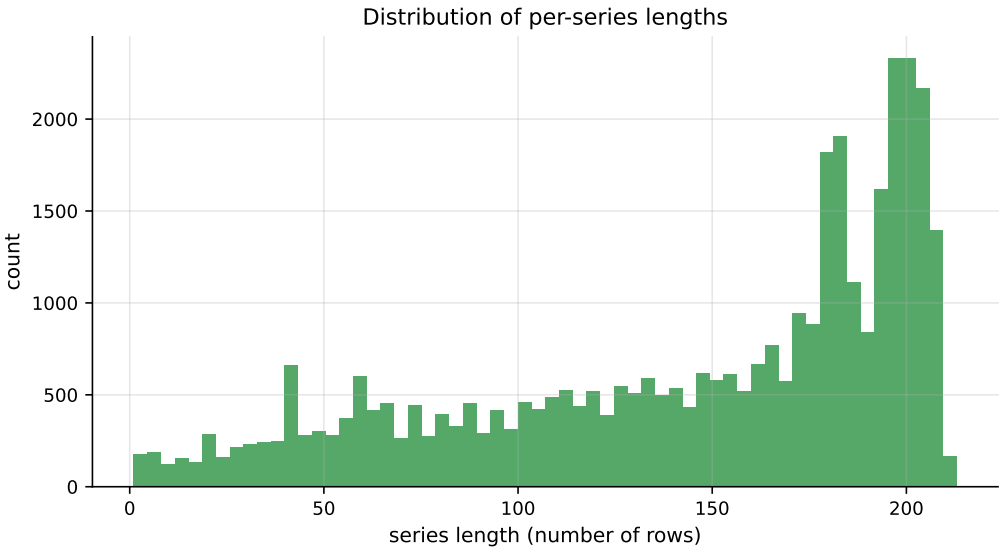

Figure 4: Distribution of per-series lengths. Most series are short, and the tail of long series carries the structural signal that classical fits can exploit.

these four panels is one of the cleanest diagnostics for whether a model family generalises or only fits one regime.

4.7 Stationarity diagnostics

Classical ARIMA-family forecasting assumes stationarity, or stationarity-after-differencing. We therefore run the Augmented Dickey–Fuller test on a stratified sample of 200 eligible series. The ADF test takes the null hypothesis that the series has a unit root (non-stationary); we reject at the 5% level when the p-value is below 0.05. Figure 6 summarises the results.

Approximately 70.5% of sampled series reject the unit-root null at the 5% level. The picture is more nuanced when broken down by horizon: H1 series are more often stationary than longer-horizon cumulative series, which is consistent with the multi-horizon coupling we will document in Section 4.14.

ADF and KPSS have opposite null hypotheses, so running both gives a more honest joint verdict: ADF takes non-stationarity as the null, while KPSS takes stationarity as the null. We classify a series as *stationary* when ADF rejects and KPSS does not, as *unit root* when ADF fails to reject and KPSS rejects, and as *inconclusive* otherwise. Figure 7 reports the joint distribution.

The stationary share dominates, but the inconclusive bucket is large enough that one-size-fits-all claims about raw levels would be unsafe. We need a repair step (differencing) for the unit-root and inconclusive cases.

Figure 8 re-runs the ADF test on the first-differenced series. The stationary share rises from 70.5% on raw levels to 98.5% after differencing.

Reading. The diagnostic chain motivates ARIMA with $d = 1$ as the default for the classical study and justifies engineered differenced features for global ML models. The 98.5% post-difference stationarity also bears on the cumulation finding in Section 4.14: differencing a higher-horizon target partially inverts the rolling-sum construction back into the H1 process.

4.8 Visual stationarity — rolling statistics

Statistical tests can disagree on short or noisy series, so a rolling-window view is a useful visual check. After first differencing, the rolling means of the four representative series hug zero and

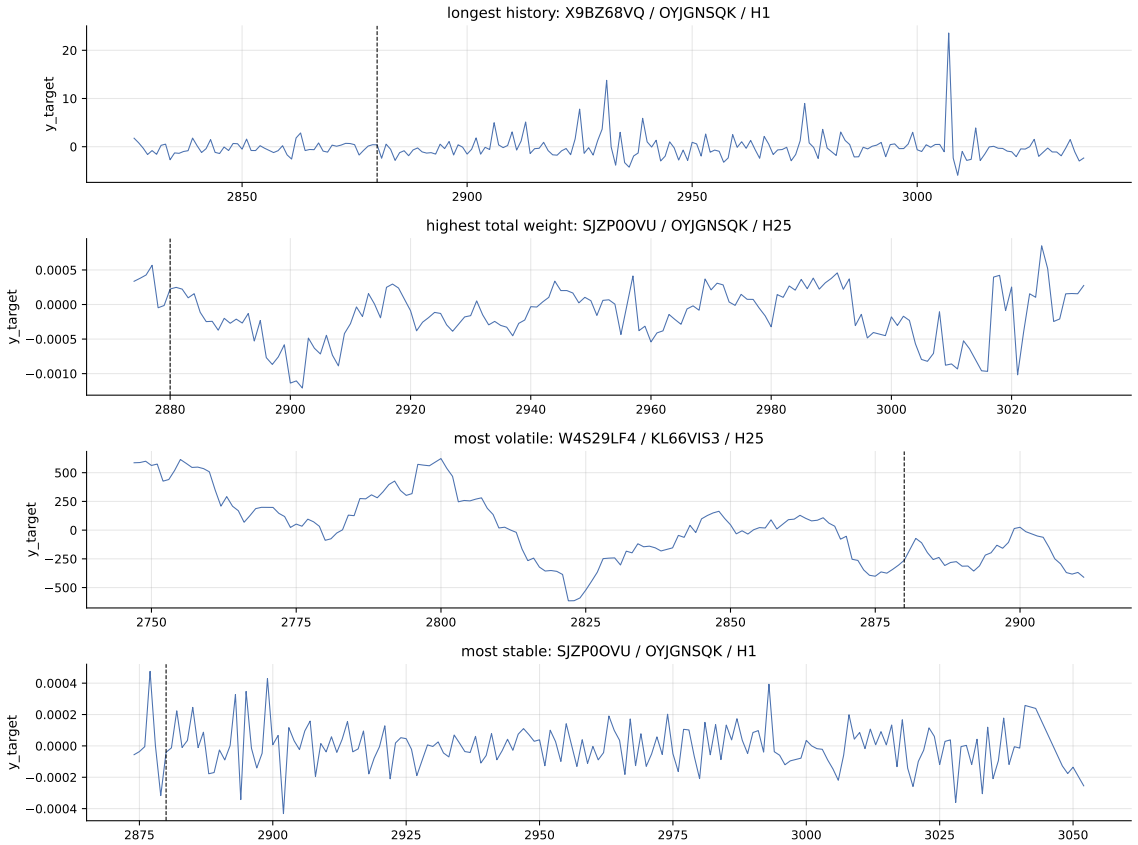

Figure 5: Raw y_{target} for the four chosen representative series. The dashed vertical line marks `ts_index` = 2880, the train/validation cutoff. The four panels span the four stress regimes that any model has to handle.

their rolling standard deviations flatten across the cutoff (Figure 9) — visually confirming the statistical jump from the previous subsection. The corresponding raw plots, omitted here for brevity, show the opposite: drifting means and bulging variances on the most-volatile series.

Reading. Differenced series behave like the white-noise regime we want before fitting a differenced classical model.

4.9 Lag structure — ACF and PACF

The autocorrelation function (ACF) shows correlation between the series and lagged copies of itself; the partial autocorrelation function (PACF) isolates each lag's direct contribution after controlling for shorter lags. A sharp PACF cutoff suggests an autoregressive order; a sharp ACF cutoff suggests a moving-average order. Rather than relying on a single anchor series, we aggregate the absolute ACF and PACF across all 200 sampled eligible series (Figure 10). The panel-level view exposes which lags are reliably informative across the panel rather than artefacts of one trajectory.

Reading. Short-memory dynamics dominate (lags 1–3 carry most of the structure with rapid decay thereafter), and there are no clean seasonal bumps. The lag feature set for the global ML models therefore prioritises lags 1, 2, 3, 5, 10 and skips seasonal indicators.

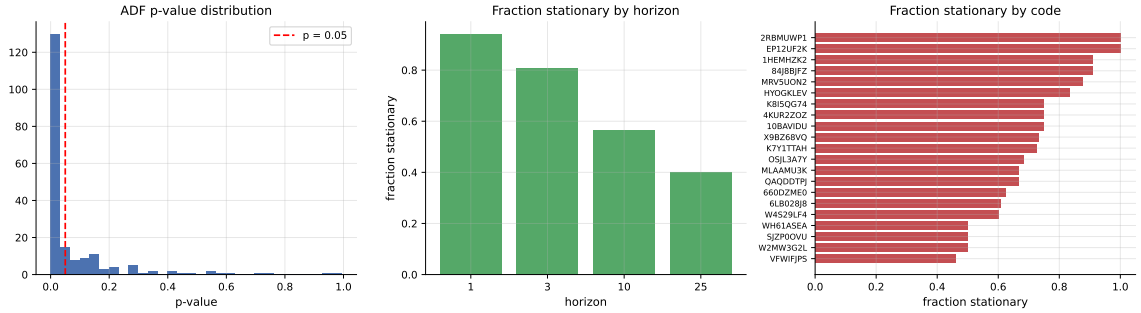

Figure 6: ADF results on 200 sampled eligible series. Left: p-value distribution with the 0.05 threshold marked. Centre: fraction stationary by horizon. Right: fraction stationary by code.

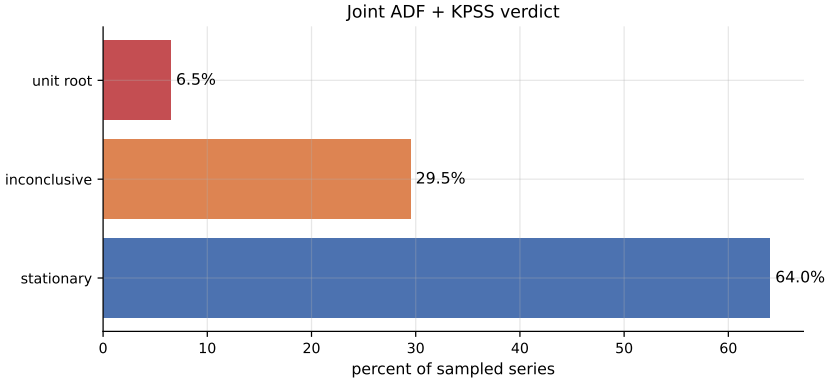

Figure 7: Joint ADF + KPSS verdict on 200 sampled series. Stationary 64.0%, inconclusive 29.5%, unit root 6.5%.

4.10 STL decomposition

Seasonal-Trend decomposition using LOESS (STL) splits a series into trend, seasonal, and residual components. We apply STL to the longest-history anchor with a period of 24 and observe the relative amplitudes of the three components. Figure 11 shows the result.

Reading. The trend captures slow drift, but the seasonal amplitude is small relative to the residual. Combined with the absence of seasonal bumps in the panel ACF, this confirms periodic structure is not the dominant signal — modelling effort goes into short-memory dynamics, not seasonal indicators.

4.11 Feature audit

The 86 `feature_*` columns are anonymised and we cannot interpret them by name. We can, however, audit them statistically. On a 200,000-row stratified sample we compute Pearson correlation between each feature and y_{target}, then rank features by absolute correlation. Figure 12 shows the top twenty.

Reading. The single largest absolute correlation is below 0.10. No single feature predicts the target well on its own. A separate correlation check among the top twenty features further reveals strong feature–feature blocks, indicating real redundancy in the engineered set. Together these say: the signal is weak, spread across many correlated features, and best exploited by gradient-boosted trees combined with caution when reading importance plots.

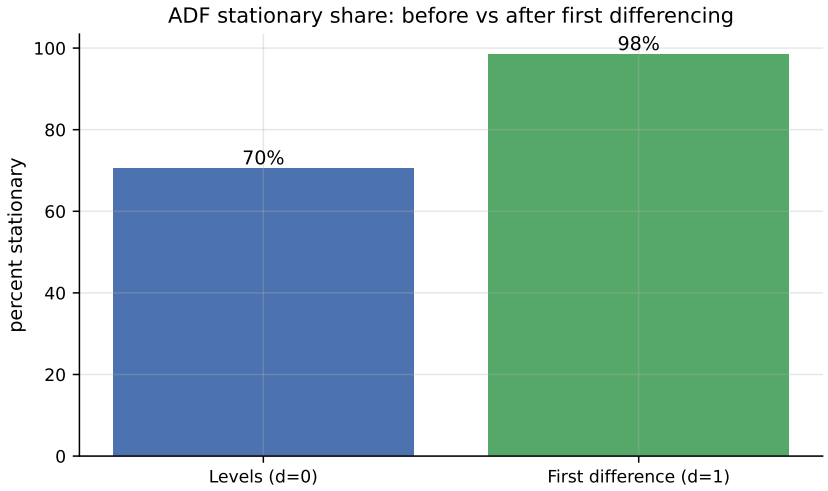

Figure 8: ADF stationary share at the 5% level: levels (left, $d = 0$) versus first differences (right, $d = 1$). Differencing pushes the share to nearly 100%.

4.12 Cross-series relationships

Within the same `code`, different sub-codes might share signal. If they do, global pooled models can borrow strength across related sub-codes; per-series local models cannot. We compute lagged cross-correlations between random pairs of sub-codes within the same `(code, horizon)` group. Figure 13 shows six representative pairs.

Reading. Clear lead/lag peaks within a code mean global models can learn a shared cross-sub-code component that per-series fits cannot. This is one of the strongest a-priori arguments for the global ML approach.

4.13 Multi-horizon coupling

The midway report treated (`code`, `sub_code`, `sub_category`, `horizon`) as the natural series identifier and fit each horizon independently. The exploratory analysis in this project shows that the four horizons of the same triple are not independent. For approximately 1.21 million distinct (`code`, `sub_code`, `sub_category`, `ts_index`) tuples, all four horizons are simultaneously present.

A second consequence of multi-horizon coupling is per-horizon target scaling. Figure 14 shows the target standard deviation and maximum row weight by horizon.

The target standard deviation rises from approximately 11.7 at H1 to 52.8 at H25, a factor of 4.5.

Reading. This finding has direct modelling consequences. A single model trained on horizons pooled into one DataFrame, without horizon-aware standardisation, will be dominated by H25 magnitudes and predict at H25 scale. Predictions at H25 scale evaluated against H1 targets will overshoot by a factor of roughly 4.5, producing catastrophically large weighted RMSE on H1 rows. This is exactly the failure mode observed in earlier ML artefacts attached to the project — the original XGBoost run reported a weighted RMSE more than two hundred times the naive baseline, consistent with a per-horizon-scale leak. The remedy is straightforward: train one model per horizon, or normalise the target by horizon before training.

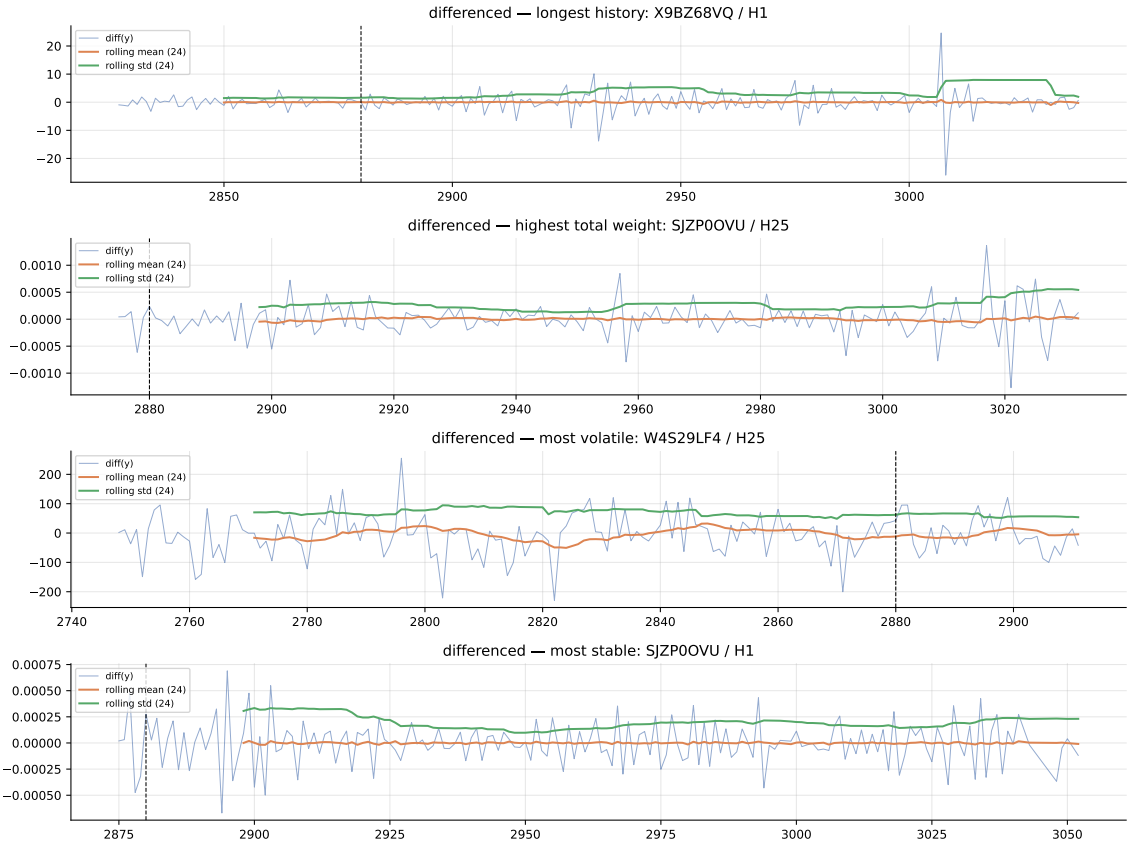

Figure 9: Rolling mean and rolling standard deviation (window = 24) on the first-differenced four representative series. The mean oscillates close to zero and the variance is stable, consistent with stationarity.

4.14 Cumulation hypothesis

Given the multi-horizon coupling above, the natural hypothesis about how the four horizons relate to each other is that the target at horizon k is a cumulative aggregate of the H1 target over the next k steps:

$$y_t^{H_k} \approx \sum_{i=0}^{k-1} y_{t+i}^{H_1}. \tag{2}$$

We test Equation (2) on a stratified sample of 500 triples that have all four horizons and at least 60 H1 rows. For each triple we pivot the data so that horizons sit on columns, compute the rolling sum of the H1 series over the appropriate window, and report the Pearson correlation between $y_t^{H_k}$ and that rolling sum. Figure 15 shows the distribution of correlations and the median ratio.

Table 2 reports the summary statistics. At $k = 3$ the median Pearson correlation is 0.96 and the median ratio is 1.00, indicating essentially exact aggregation. The relation is slightly noisier at $k = 10$ and $k = 25$, with median ratios of 0.95 and 0.91, but still clearly dominant.

Reading. Two consequences for the modelling work that follows. First, fit-H1-and-aggregate becomes a viable, possibly superior strategy to fitting per-horizon models independently: a single H1 model gives H3, H10, and H25 forecasts for free, and the aggregation is mathematically forced by the data construction. We test this in a dedicated H1-aggregation experiment in a later notebook. Second, differencing operations on $y^{H_{25}}$ partially recover the y^{H_1} process, which explains why ARIMA with $d = 1$ wins the most-volatile H25 series in the midway classical study:

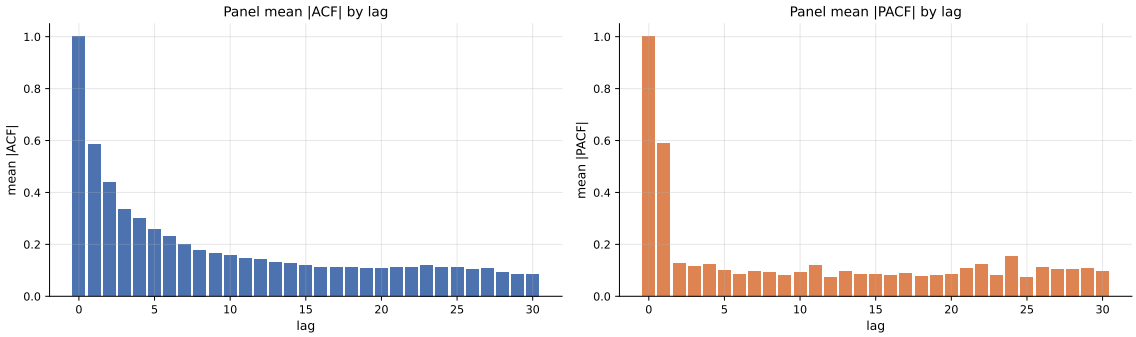

Figure 10: Panel-level mean |ACF| (left) and mean |PACF| (right) across 200 sampled eligible series. Persistent low-order lags dominate; higher lags decay quickly. There are no clean seasonal bumps.

Table 2: Cumulation hypothesis test summary on 500 triples.

k	n triples	mean corr	median corr	median ratio
3	500	0.92	0.96	1.00
10	500	0.86	0.94	0.95
25	500	0.78	0.91	0.91

differencing inverts the cumulation. The midway interpretation that ARIMA was capturing a difficult pattern is partly correct; it was also undoing the cumulative construction.

4.15 Per-horizon weight dominance

The weight distribution analysis in Section 4.4 was panel-wide. Because `weight` and `horizon` are correlated, the rows that drive the weighted skill score may not be evenly distributed across horizons. Figure 16 reports the share of total panel weight per horizon and the share of weight within the top 1% of all weight mass.

The headline numbers are striking. The maximum panel weight (1.39×10^{13}) is at H3, not H1. H3 alone holds 37.4% of total panel weight and 43.6% of the top-1% weight mass; H1 holds 25.4%, H10 holds 14.3%, and H25 holds 16.8%.

Reading. A model that predicts H3 well on the small set of high-weight rows will tend to win the leaderboard regardless of how it performs on the other horizons. This was missed in the midway report. The practical consequence is that we report H3 weighted skill as a separate headline number alongside the panel-wide score, and we track H3 performance carefully when comparing models.

4.16 EDA-driven modelling implications

The EDA constrains modelling along several axes. Table 3 summarises the constraints and the corresponding decisions for later notebooks.

These constraints shape every modelling chapter that follows. The classical baselines (notebook 01) honour the chronological cutoff and the four representative series. The global LightGBM models (notebook 02) train per horizon with sample weights. The H1-aggregation experiment (notebook 03) operationalises the cumulation hypothesis. The probabilistic forecasting work (notebook 04) addresses the heavy tails and weak per-feature signal directly. The ensembles (notebook 05) fit weights on a meta slice that respects chronology. The final comparison (notebook 06) reports both panel-wide and H3-specific weighted skill scores with bootstrap

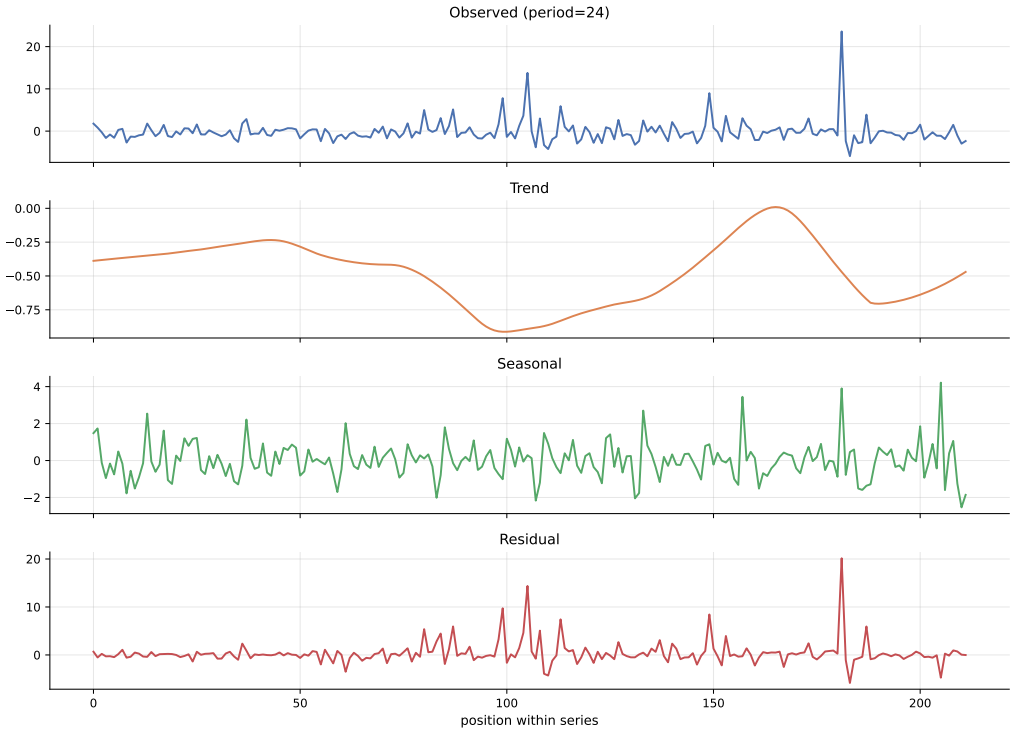

Figure 11: STL decomposition of the longest-history anchor with period 24. The trend component captures slow drift; the seasonal amplitude is small relative to residual standard deviation.

intervals.

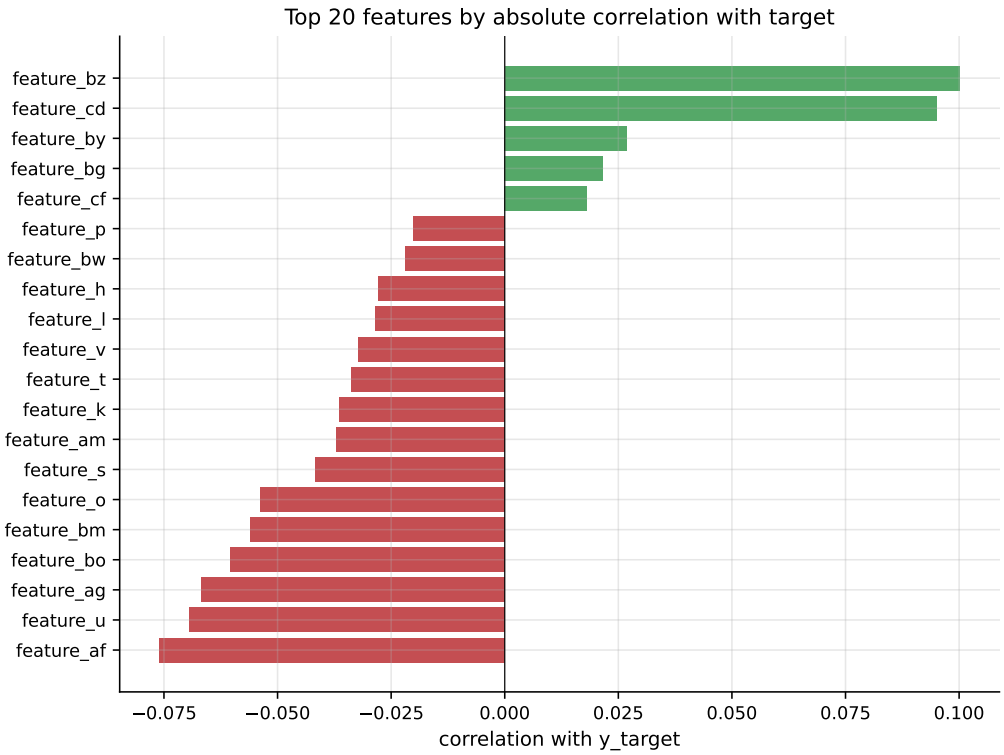

Figure 12: Top twenty features by absolute correlation with y_{target}, computed on a 200,000-row stratified sample. Green bars are positive correlations, red bars are negative.

Figure 13: Lagged cross-correlation between random sub-code pairs sharing the same code and horizon. A peak away from lag 0 means one sub-code leads or lags the other.

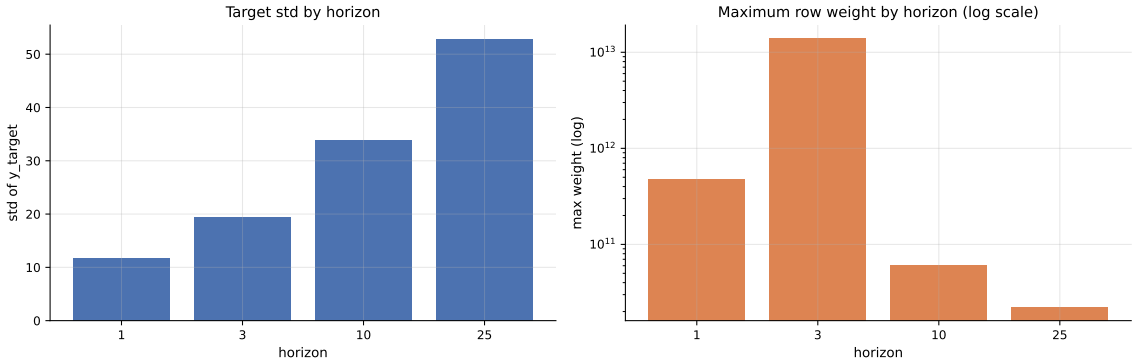

Figure 14: Per-horizon target standard deviation (left) and maximum row weight on a logarithmic scale (right). Target scale grows by a factor of approximately 4.5 from H1 to H25. Maximum weight is highest at H3.

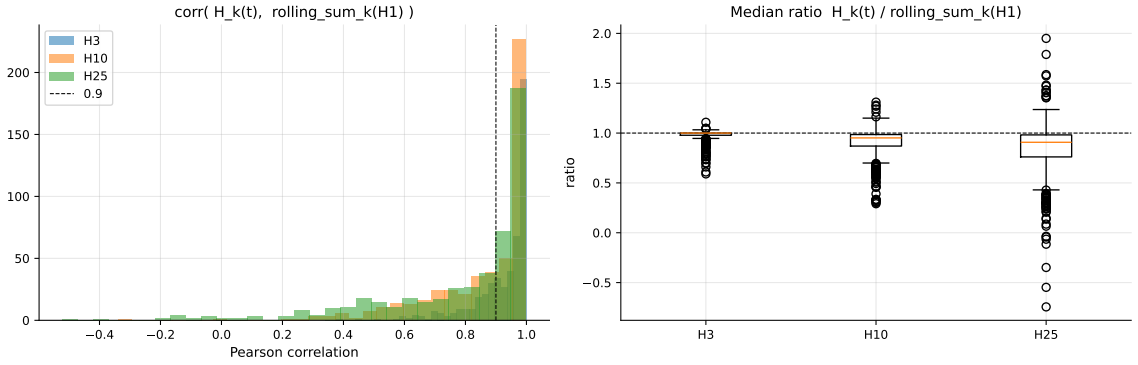

Figure 15: Test of the cumulation hypothesis on 500 stratified triples. Left: distribution of Pearson correlations between $y_t^{H_k}$ and the rolling sum of y^{H_1} over k steps, for $k \in \{3, 10, 25\}$. Right: distribution of the per-triple median ratio $y_t^{H_k} / \sum y^{H_1}$. Higher and tighter is better.

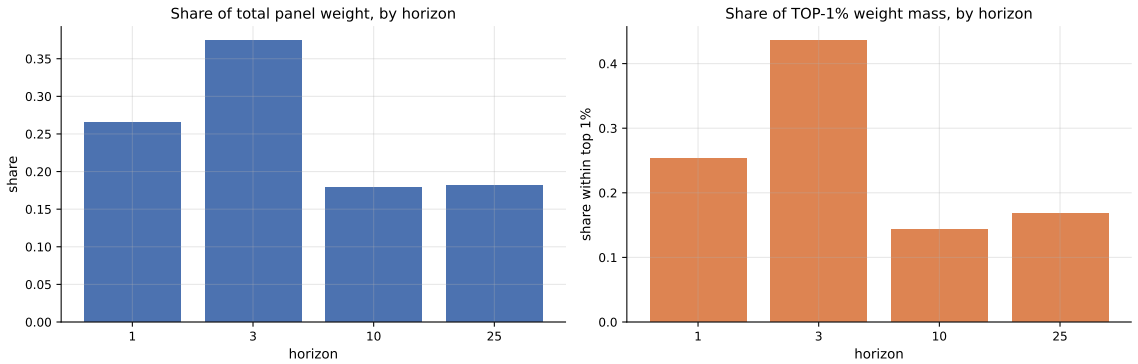

Figure 16: Per-horizon weight breakdown. Left: each horizon's share of total panel weight. Right: share of the top 1% of weight mass by horizon. H3 dominates both views.

Table 3: From EDA findings to modelling decisions.

EDA finding	**Modelling decision**
Test is a strict temporal suffix	Validation must be chronological. No random CV folds.
Top 1% of rows hold $\sim 64\%$ of weight	Sample-weight every loss; never report unweighted RMSE as headline.
Heavy target tails (± 2200) with IQR ± 0.05	Robust losses (Huber, quantile) for any model on raw y_{target}.
70% raw stationary, 99% after differencing	ARIMA with $d = 1$ default; differenced features for ML.
Persistent low-order ACF/PACF; weak STL seasonality	Lag features 1, 2, 3, 5, 10; no seasonal indicators.
Cross-series correlation within codes	Global pooled models over per-series fits.
Multi-horizon coupling and cumulation	Fit-H1-and-aggregate as a candidate model class. Per-horizon training as a baseline.
Per-horizon target scaling differs by $4.5\times$	One model per horizon, or horizon-aware target standardisation.
H3 dominates top-1% weight mass	Report H3 weighted skill as a separate headline.
Max $\lvert\text{corr}\rvert$ across features < 0.10	Gradient-boosted ensembles preferred over linear models on raw features.
Strong feature–feature correlation in top features	Read importance plots with collinearity in mind.